

Avaliação Técnica

Teste Técnico – Plataforma de Gestão de Frota

Tipos: Backend · Frontend · Fullstack

1. Problema – Gestão de Frota

A **Aivacol** é uma solução de gestão inteligente para locadoras, oferecendo automação, rastreamento, análise de frota e integração entre módulos operacionais. Sua tarefa é implementar parte do módulo responsável pelo **ciclo de vida de veículos**, garantindo que a plataforma seja:

-  **Segura**
-  **Escalável**
-  **Eficiente**
-  **Organizada e de fácil manutenção**
-  **Testada e confiável**

O domínio envolve:

- **Veículos**
- **Modelos**
- **Marcas**
- Controle de operações e histórico
- Auditoria (opcional)
- Integração via mensageria (opcional)

Antes de começar, selecione o tipo de avaliação:

 **BACKEND**  **FRONTEND**  **FULLSTACK**

2. Modelagem de Dados (Tabelas em Inglês)

(mantida conforme seu documento)

2.1. Tabela `brands`

Campo	Descrição
<code>id</code>	Identificador da marca
<code>name</code>	Nome da marca
<code>created_at</code>	Data de criação
<code>updated_at</code>	Data de atualização
<code>created_by</code>	Usuário responsável

📁 2.2. Tabela `models`

Campo	Descrição
<code>id</code>	Identificador do modelo
<code>name</code>	Nome do modelo
<code>brand_id</code>	FK para <code>brands</code>
<code>created_at</code>	Data de criação
<code>updated_at</code>	Data de atualização
<code>created_by</code>	Usuário responsável

📁 2.3. Tabela `vehicles`

Campo	Descrição
<code>id</code>	Identificador
<code>license_plate</code>	Placa
<code>chassis</code>	Chassi
<code>renavam</code>	Renavam

<code>year</code>	Ano
<code>model_id</code>	FK para <code>models</code>
<code>created_at</code>	Criado em
<code>updated_at</code>	Atualizado em
<code>created_by</code>	Responsável

2.4. Tabela `users`

Campo	Descrição
<code>id</code>	Identificador
<code>nickname</code>	Nome curto
<code>name</code>	Nome completo
<code>email</code>	Email

Mock Obrigatório (`seed_vehicles.json`)

(mantido conforme documento)

BACKEND – Requisitos

Objetivo

Construir o backend do módulo de Gestão de Frota garantindo:

- Arquitetura limpa
 - Segurança robusta
 - Testes automatizados
 - Escalabilidade
 - Padronização da modelagem
-

Tecnologias Selecionadas

- **Node.js** (+18)
 - **NestJS** (preferencial +10)
 - **TypeORM**
 - **SQL Server**
 - **JWT**
 - **Jest** (testes obrigatórios)
 - **Redis para Cache** (novo 🚀)
-

Requisitos Funcionais

Gestão de Brand (`brands`)

- Criar
- Atualizar
- Consultar
- Remover

Gestão de Model (`models`)

- Sempre associado a Brand
- Criar, atualizar, consultar, remover

Gestão de Vehicle (`vehicles`)

- Registrar
- Atualizar
- Listar / consultar
- Remover

Segurança

- Autenticação JWT
- Todas as rotas acima devem ser protegidas
- Usuário padrão: **aivacol**

Metadados obrigatórios

- `created_at`
- `updated_at`

- `created_by`

Testes (obrigatórios)

Cobrir:

- regras de negócio
 - serviços
 - validações
 - integrações mínimas
-

Tecnologias opcionais (BÔNUS)

Mensageria

- RabbitMQ (preferencial)
- Kafka
- SQS

Auditoria

- MongoDB ou DynamoDB

Registre todas as interações do serviço backend no banco não relacional para fins de auditoria (BÔNUS)

Docker / Docker Compose

- Dockerfile multistage (BÔNUS)

Cache com Redis

- Cache de consultas de veículos
 - Expiração configurável
 - Invalidação automática em alterações
-

FRONTEND – Requisitos

Objetivo

Construir a interface de Gestão de Frota utilizando **Angular** com:

- UX clara
- Componentização

- Integração com APIs ou mocks
- Organização sustentável

 *O candidato frontend pode mockar como preferir: serviços fake, JSON local, interceptors simulados etc.*

Tecnologias obrigatórias

- Angular 16+
 - TypeScript
 - RxJS
 - Routing
 - Interceptors
-

Funcionalidades obrigatórias

Tela de Login

- Com reatividade ou mock
- Guard para rotas internas
- Armazenamento de token

Lista de Veículos

- Tabela ou cards
- Exibir:
 - license_plate
 - brand
 - model
 - year
- Buscar via backend ou mock

Cadastro/Edição de Veículo

- Formulário validado
- Dropdown de brand/model
- Consumo de API ou mock

Estrutura

- Componentização

- Serviços bem definidos
 - Pastas organizadas
-

FULLSTACK – Requisitos

Objetivo

Implementar **todo backend obrigatoriamente + as telas essenciais do frontend**:

- Login
- Lista de veículos
- Cadastro/Edição

Será valorizado domínio equilibrado entre as duas áreas.

7. Critérios de Avaliação

Critérios Gerais (para todos)

- Clareza do código
 - Redundância e Eficiência
 - Organização do projeto
 - Estrutura de pastas
 - Boas práticas
 - Aderência ao problema
 - Documentação técnica
 - Qualidade do README
-

BACKEND – Critérios específicos

- Arquitetura e modularização
- Uso correto do TypeORM
- Migrations bem definidas
- Segurança com JWT
- Implementação do Redis Cache
- Testes Jest cobrindo corretamente
- Qualidade das regras de negócio

- Tratamento de erros e exceções
 - Boas práticas REST
-

FRONTEND – Critérios específicos

- UX e clareza da navegação
 - Componentização
 - Uso de interceptors
 - Boas práticas Angular
 - Validação dos formulários
 - Integração com API ou mocks
 - Organização de serviços e módulos
-

FULLSTACK – Critérios específicos

- Integração real frontend ↔ backend
 - Consistência do domínio nos dois lados
 - Coesão estrutural
 - Entrega completa e funcional
-

8. Entrega

O candidato deve entregar:

- Repositório no GitHub
- Mock incluído no repositório
- README com instruções
- Scripts de execução
- Testes (para backend obrigatoriamente)
- Caso fullstack: instruções para rodar todo o ambiente